1. Purpose and observed outcome

ScopeLock Verify | Hack Apertus 2026 | Track 2B | Agent3 / Joey

A video can match a duration while still failing the brief. Reviewers also face obsolete requirements, reference files and quoted dialogue that look like technical specifications. ScopeLock links original text, explicit reviewer decisions and actual file observations, while keeping unverified requirements visible.

This is a new research prototype started on 15 October 2026. The working local application implements a complete source-review-to-evidence path. Apertus is used for advisory span classification. The original thinking protocol failed its semantic gate. A repaired official-template proposition protocol passed a separate small frozen internal set (12/12 candidates, 6/6 briefs). This does not establish production reliability, autonomous acceptance or demonstrated productivity improvement.

What runs	What it establishes
Lexical source candidates	Exact original quotations, offsets and separate normalized values. No semantic completeness.
Explicit review confirmation	Reviewer-selected technical values, line dispositions and tolerance. Local attribution, not authenticated human identity.
Bounded ffprobe	Technical observations against the selected specification only.
Version/digest-bound report	Inconsistent source, specification or file invalidates current evidence. Not a signed external attestation.

Example shown in the demo: the brief requests MP4 at exactly 2 seconds, withdraws an old 5-second reference and leaves tone/subtitles for manual review. A real synthetic 2-second file matches the selected fields; a 1-second file mismatches duration; corrupt bytes are UNVERIFIED. Tone and visible subtitles never turn green.

All demo interactions and generated files are synthetic controls. Automated browser clicks are labeled SCRIPTED TEST. There is no customer deployment, actual human study or measured time-saving percentage.

2. Architecture and trust boundaries

Target architecture: on-premise. A single local container holds the Python workbench, bounded FFmpeg probe and pinned llama.cpp server. The supplied Apertus GGUF is mounted read-only, verified by SHA-256 and loaded on loopback only. The workbench is exposed on host loopback; source text and media do not require a cloud inference API.

Flow: original source -> lexical candidate spans -> optional untrusted Apertus advice -> independent explicit review action -> selected technical specification -> real file probe -> per-field MATCH / MISMATCH / UNVERIFIED -> current evidence JSON. Model advice does not select candidates, resolve conflicts or set approval state.

Every candidate needs select/exclude/unresolved. Every complete source line needs a disposition and explanation, including unsupported instructions. Choosing technical-reviewed means the reviewer reviewed that line; it is not proof that all meanings have been recognized. Unresolved candidates and manual-review lines remain in the report.

Changing source increments its version and clears old advice, confirmation and reports. Editing a review draft disables inspection/export and invalidates the stored confirmation. Confirmation binds source hash/version, selections, coverage, reviewer and explicit tolerance. The server recomputes the specification from the actual source-bound candidates. A new specification requires a new file observation.

Each file is read and probed again, including a new file with identical dimensions/duration. Export rereads its SHA-256 and checks report consistency. Full raw model traces stay private; exported advice includes only allowlisted provenance and labels. Hashes detect inconsistency; a malicious local administrator can rewrite the entire state consistently.

The local HTTP app enforces Host, same-origin and per-process CSRF checks. It is not a multiuser service and does not authenticate the physical reviewer. An agent controlling the browser can operate review controls. The security boundary is separation of application operations, not proof of human presence.

Probe limits: regular files beneath a private upload root, 64 MiB input, allowed demuxers/file protocol only, eight-second wall timeout, CPU/output/allocation limits. Ambiguous container brands, multiple videos, missing timing and unsupported display matrices fail closed. These bounds are not a complete OS-level media sandbox.

3. Evaluation and negative evidence

Legacy 8/28 datasets and the narrow English rule baseline were frozen for the original thinking run and were not retuned during that run. For repair, the old 8 were debugging cases only. A separately authored new 12 set was frozen before protocol selection and run once after selection. These are internal synthetic sets, not external blind or human studies.

Evaluation	Measured result
Repaired protocol / new 12	Apertus: 12/12 labels, 6/6 exact briefs; 0 unsafe KEEP; 0 UNAVAILABLE. Rule: 7/12 labels, 2/6 briefs. Lexical recall12/12. Median wall time 16.3s; internal synthetic single run, not external or human validation.
Legacy: independent complex v1	Apertus: 16/28 candidate labels; 0/8 exact briefs; 2 unsafe KEEP suggestions; 3 UNAVAILABLE. Rule: 22/28 labels, 4/8 exact briefs. Median wall time 65.6s per span on CPU2 under recorded concurrent host load (not an isolated hardware benchmark).
Legacy: original span8 regression	Apertus: 4/8 candidate labels; 1/4 exact briefs; 1 unsafe KEEP suggestions; 1 UNAVAILABLE. Rule: 5/8 labels, 2/4 exact briefs. Median wall time 77.2s per span on CPU2 under recorded concurrent host load (not an isolated hardware benchmark).
Engineering tests	51 test methods passed on the recorded native release run; independent actual file and HTTP controls.
Real media control set	13/13 saved controls. Includes an injected missing-observation case, separately labeled; not 13 independent real files.
Legacy complex extraction	26/28 annotated spans found; 12/12 KEEP spans; 6/8 complete annotated span sets. Misses: a unitless range bound and French seconds. Not semantic or whole-source coverage.

Apertus chooses KEEP or REVIEW for a supplied exact source span. Scoring uses exact candidate labels, exact per-brief sets, unsafe KEEP on REVIEW gold, missed KEEP and UNAVAILABLE. Unsafe KEEP is a wrong model suggestion, not an actual file acceptance. Classification on provided spans is separate from candidate extraction recall.

The current adapter sends system/user messages to the official llama.cpp b11429 chat endpoint, uses the GGUF chat template with enable_thinking=false, and asks one explicit natural-language proposition about one field/value in the complete source. A nested JSON schema restricts required to a boolean; strict parsing rejects duplicate keys, wrong roles, extra keys, tools/reasoning and incomplete output. Grammar controls do not guarantee semantic truth. The old native-token parser is retained only for historical reproduction.

Earlier, separate research reported 19/24 labels and 8/12 exact briefs (normalized candidates), then 5/8 labels and 1/4 briefs (original spans); both had one unsafe KEEP. These runs are not pooled. Their CPU/cache/protocol settings differ.

Debugging was limited to two protocols on old 8: A 5/8 with 2 unsafe KEEP; B 8/8 with 0. B was frozen before opening or running the separately authored new 12. No third protocol or holdout retuning occurred. New12 corrected 5 rule errors and broke 0 rule-correct cases. Multiple factors changed, so neither a sole cause nor generalized AI benefit is established. The interface preserves identical review controls with either hint source. We measured classifier decisions, not reviewer time; there were no human participants or gold-answer substitution. Integrity improvements are not attributed to AI.

4. Reproducibility, limits and disclosure

Recorded model/runtime acceptance: e867935c8c9cceac4d56b6c538b8a12a1186ac3f. Later configuration changes have separate stub-based evidence, not new model acceptance.

From repository root, make run uses APERTUS_MODEL for the local GGUF. LLM_NAME sets the local model alias; LLM_BASE_URL sets its loopback endpoint; optional LLM_API_KEY enables server authentication and adapter Bearer authorization. APERTUS_URL remains a legacy endpoint alias. Invalid/remote endpoints and redirects are refused; keys are excluded from traces and arguments. Build needs Docker and network downloads; runtime uses local weights/media. No weights are silently downloaded; run-manual is a diagnostic only.

Deployment evidence: A clean Linux arm64 clone at the accepted runtime commit started with the default root make run. Linux 51/51 passed. One actual chat API control returned a strict boolean and completed its file/approval/invalidation lifecycle in 61.863 seconds. A separate actual-browser model control preserved in-flight manual edits and created no approval; matching, mismatching and damaged files plus source-invalidation/export controls passed on the same source. Earlier native-token and b09 evidence remains historical. These are synthetic deployment/integrity controls, not semantic generalization or human studies. The ordinary on-premise bridge retains outbound capability; no airgap claim is made. See docs/docker-protocol-repair-summary.json for source/image hashes and negative controls.

Model: community Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF, revision 248ec68a63e219e3f061e4c946fc8a20d63b9975. Public Hugging Face LFS SHA-256 exactly matches the local file: a037df8d87ff6caacee794ee85f55342f2152e0d359b7389033300c3bee5f299 (5,059,027,136 bytes). The publisher declares conversion from andreasmartin/apertus-v1.5-8b-text; the upstream family is swiss-ai/Apertus-v1.5-8B. The official family discussion links this community quantization. We identify a downloadable exact artifact, not an official release or independently rebuilt conversion. See docs/model-source.json for the pinned URL.

Runtime: official llama.cpp b11429, source d81235049384534c167caea52b85a694f6103d14; CPU two threads, two batch threads, one parallel slot, context 4096, GPU off. Current protocol: temperature zero, seed 42, at most 64 output tokens, thinking disabled, strict JSON boolean. The historical thinking protocol used at most 768 tokens and native IDs 32/33/68 for this file only.

Local tests: PYTHONPATH=track_2b python3 -m unittest discover -s track_2b/tests -v. The configuration revision passed 64 native tests, including 13 transport controls using local HTTP stubs, process environment checks and mocked container startup; no model or Docker acceptance was rerun. Frozen evaluation bytes and semantic request settings remain unchanged. Public traces are opt-in; private conversations/customer media are excluded.

Development used the OpenAI Codex coding assistant for implementation, synthetic fixtures and documentation. Apertus is the only application inference model. No alternate model or human gold answer substitutes for its predictions. Organizer template and code retain Apache-2.0; report uses CC-BY-4.0; project-authored synthetic texts and gold annotations use CDLA-Permissive-2.0 (prior CC-BY-4.0 alternative retained). Dependencies retain their licenses. No new model AUP, event terms or publication agreement was accepted during this local build.

Open limits: broader semantic reliability and generalized incremental value not established; the repaired protocol has not rerun the legacy 28 complex set; small frozen12 success is not production evidence; candidate extraction omits unsupported expressions; no general semantic coverage; no production human evaluation; local approval is not authenticated; no public deployment hardening. Next step is a controlled reviewer study only after a stronger, independently evaluated model protocol and verified deployment. Community conversion has not been independently rebuilt.

References: organizer template https://github.com/HackApertus/project-template/tree/main/track_2b ; official deadline/rules https://hackapertus.devpost.com/rules ; model card https://huggingface.co/swiss-ai/Apertus-v1.5-8B ; llama.cpp server https://github.com/ggml-org/llama.cpp/tree/b11429/tools/server ; ffprobe https://ffmpeg.org/ffprobe.html